ECG SIGNAL PROCESSING FOR ARRHYTHMIA DETECTION

A project report submitted in partial fulfillment

of the requirements for the degree of

Bachelor of Technology

in

Electronics & Computer Engineering

By

V M ADHITYAA - 21BLC1058

VINODH S - 21BLC1472

A NAVANETH - 21BLC1569

School of Electronics Engineering,

Vellore Institute of Technology Chennai,

Vandalur-Kelambakkam Road,

Chennai - 600127, India.

November 2024

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Declaration

I hereby declare that the report titled "**ECG SIGNAL PROCESSING FOR ARRHYTHMIA DETECTION**" submitted by me to the School of Electronics Engineering, Vellore Institute of Technology, Chennai in partial fulfillment of the requirements for the award of **Bachelor of Technology** in **Electronics and Computer Engineering** is a bona-fide record of the work carried out by me under the supervision of **Dr. Ranjeet Kumar**.

I further declare that the work reported in this report, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Sign: _____

Name & Reg. No.: _____

Date: _____

Sign: _____

Name & Reg. No.: _____

Date: _____

Sign: _____

Name & Reg. No.: _____

Date: _____

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Electronics Engineering

Certificate

This is to certify that the project report titled **ECG SIGNAL PROCESSING FOR ARRHYTHMIA DETECTION** submitted by **V M Adhityaa (21BLC1058), Vinodh S (21BLC1472), A Navaneth (21BLC1569)** to Vellore Institute of Technology Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in **Electronics and Computer Engineering** is a bona-fide work carried out under my supervision. The project report fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor		**Head of the Department**	
Signature:	.....	Signature:	.....
Name:	.....	Name:	.....
Date:		Date:	

Examiner

Signature: .....

Name: .....

Date:

(Seal of the School)

Abstract

Cardiovascular diseases are one of the leading causes of human death globally, with millions affected each year. Early detection of CVDs is crucial for effective treatment and management. Conventional detection methods might be effective but those methods are difficult, slow, expensive and not suitable for frequent measurement and diagnosis. Automatic detection of abnormal beats from ECG can be helpful in the early detection, diagnosis and prevention of CVDs. Several Deep Learning methods have recently gained attention in the field of healthcare to help automate the detection of abnormalities in ECG signals.

ECGs are still the benchmark for assessing the electrical activity of the heart and abnormalities in ECG are the signs of onset of CVDs. Several deep learning models, including CNN and RNN have shown great promise in health applications for detecting arrhythmias and other abnormalities from ECG signals. In this paper we focus on developing a hybrid model by Convolutional Neural Networks (CNN) with sequence processing models like Bidirectional Long Short-Term Memory (BiLSTM) and gated convolutions. The model is trained on the MIT-BIH Arrhythmia dataset. The proposed model aims to achieve high accuracy in detecting arrhythmia by capturing important features from both the spatial and temporal dimensions of the MIT-BIH dataset.

Acknowledgements

We wish to express our sincere thanks and deep sense of gratitude to our project guide, Dr. Ranjeet Kumar, Professor, School of Electronics Engineering, for his consistent encouragement and valuable guidance offered to us in a pleasant manner throughout the course of the project work.

We are extremely grateful to Dr. Ravishankar A, Dean Dr. Reena Monica, Associate Dean (Academics) & Dr. John Sahaya Rani Alex, Associate Dean (Research) of the School of Electronics Engineering, VIT Chennai, for extending the facilities of the School towards our project and for his unstinting support.

We express our thanks to our Head of the Department Dr. Annis Fathima A for her support throughout the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

List of Figures vii

1 Introduction 1

2 Literature Survey 2

2.1 Cardiac arrhythmia detection using deep learning approach and time frequency representation of ECG signals 2

2.2 ECG signal feature extraction trends in methods and applications 2

2.3 An Arrhythmia Classification Model based on CNN-LSTM SE Algorithm 3

2.4 Classification of Electrocardiogram Signals Based on Hybrid Deep Learning Models 3

2.5 Performance Study of Different Denoising Methods for ECG Signals 4

2.6 AECG – based cardiac arrhythmias detection through ensemble learning and fusion of deep spatial-temporal and long-range dependency features 4

2.7 ECG-TransCovNet: A hybrid transformer model for accurate arrhythmia detection using Electrocardiogram signals 4

3 Methodology 5

3.1 Dataset 5

3.2 Data Preprocessing 6

3.2.1 Denoising 6

3.2.2 R-Peak detection and Segmentation 7

3.2.3 Handling Class Imbalance and Data Splitting 9

3.3 Model Architecture 11

3.3.1 Proposed model architecture 13

3.3.2 Gated Convolution 13

3.4 Model Training.....14

4 Results and Discussions **15**

4.1 Results and comparison..... 16

4.1.1 Confusion Matrix..... 17

4.1.2 Per Class Accuracy 18

4.1.3 Performance comparison after class balancing 18

5 Conclusion and Future Scope **20**

6 Appendix **21**

List of Figures

3.1 Plot of raw ECG signal and bandpassed signal..... 6
3.2 Block Diagram of the Pan-Tompkins algorithm..... 7
3.3 Plot of differentiated and squared ECG signal..... 7
3.4 Plot of moving windows integrated signal and R-peaks annotated signal..... 8
3.5 Table showing the mapping of class labels to numbers..... ..8
3.6 Table displaying the number of samples in each class 9
3.7 Block Diagram of the proposed model 10
3.8 Architecture of the proposed model……12
3.9 Summary of the proposed model……13
3.10 Accuracy curve……14
3.11 Loss curve……14

4.1 Formulas to calculate validation metrics 16
4.2 Table displaying the accuracy results of the models..... 17
4.3 Confusion Matrix of the proposed model 17
4.4 Confusion Matrix of the CNN-LSTM model 18
4.5 Per-class score of the model……...18
4.6 Model performance comparison after SMOTE……..19

Chapter 1

Introduction

Cardiovascular diseases are the major cause of death in the world accounting for almost 20 million deaths every year. CVDs are a group of diseases that affect our heart and blood vessels by narrowing our blood vessels in our heart and other parts of the body. Several conditions of the heart such as arrhythmia, valve disease, coronary artery disease, heart failure and many other diseases can all be classified as CVDs. Arrhythmia is a condition caused by abnormal heart rhythms or heart beats due to the problems with our heart's electrical conduction system. Almost 32 percent of the global death each year is caused by CVDs. Thus, early detection and accurate diagnosis of arrhythmias are crucial for effective treatment and management of heart conditions.

ECG signals which are recordings of the heart's electrical activity, contains important information about the heart and its function. Traditional detection methods are based on the manual interpretation of ECG signals, which are time consuming, difficult, not suitable for diagnosis in a high scale and also prone to human error. Detecting conditions like arrhythmia from ECG signals is a challenging task as it requires distinguishing subtle variations in the waveform. In the recent years, with rapid advancements in machine learning and deep learning techniques, automated systems for the detection of arrhythmias have gained a lot of interest in the field of healthcare. A well-designed machine learning model can automate the diagnosis of abnormalities, thus reducing the burden on manual diagnosis and also can improve the detection accuracy.

In this proposed work, we present the development of a hybrid deep learning model that combines CNN with BiLSTM and gated convolutions. CNN is one of the most popular and straight forward DL models used for arrhythmia detection. CNN model combined with other networks to form a hybrid model has significant increase in performance and accuracy. CNN architecture is used to extract the most important features and also the spatial features from the ECG signals. BiLSTM is a sequential model and thus processes data sequentially and learns the temporal dynamics by monitoring information flow in both directions. Gated convolution introduces a gating mechanism and thus helps the model to focus on the important features while suppressing the irrelevant signals.

Chapter 2

Literature Survey

2.1 Cardiac arrhythmia detection using deep learning approach and time frequency representation of ECG signals

This research paper mainly focuses on the transition of traditional machine learning techniques to advanced deep learning models for ECG classification. It states how traditional methods relied on handcrafted features from the ECG signals such as R-R intervals and also states its drawbacks such as loss of critical information or irrelevant features being included which results in degraded performance. With advanced models such as CNN, features are no longer needed to be extracted manually.

With techniques such as wavelet transforms, 1D ECG signals can be converted to 2D images, which benefits deep learning models as they can extract important spatial features from the data. This paper also states how pre-trained models such as AlexNet and ResNet can be used to achieve better results as those models are well tuned for ECG classification.

It also references studies that show promising results with accuracies as high as 99.23% using hybrid models combining CNN and LSTMs. This paper was helpful in understanding the advancement of learning methods used for ECG classification. Our proposed work also uses deep learning methods combining CNN, BiLSTM to achieve better performance when compared to conventional machine learning techniques.

2.2 ECG signal feature extraction trends in methods and applications

This paper mainly covers the different ECG signal feature extraction methods which are primarily used for digital health and AI applications. ECG signals primarily contain PQRST waveforms. These waveforms must be extracted in a way that it reduces data complexity and optimizes the data for AI and ML applications. The aim is to extract important features that is required for efficient classification, detection and analysis and eliminating redundant information.

Due to non-linear, non-stationary nature of ECG data, feature extraction requires advanced techniques across various domains such as time, frequency and decomposition. While traditional methods are limited in application specificity, advanced methods like PCA, MAR may address signal complexity but are computationally expensive. The paper also examines DL techniques and their integration into the feature extraction pipelines.

Methods like Pan-Tompkin's algorithm and Hidden Markov models are pointed out for their success in QRS detection and arrhythmia detection. Techniques such as wavelet transformation and tensor-based methods are promising solutions for feature extraction in multi-lead ECGs. With our proposed work, CNN architecture is designed to automatically extract hierarchical features from the data. Deep learning models can bypass the need for manual feature extraction in some cases as they can learn features directly from the data.

2.3 An Arrhythmia Classification Model Based on a CNN-LSTM SE Algorithm

This paper employs an advanced hybrid deep learning model by combining CNN, LSTM and a Squeeze-and-Excitation (SE) module. It helped us understand the limitations in conventional machine learning models and also standalone deep learning models as they suffer in capturing both spatial and temporal dependencies in ECG signals. It addresses the challenges faced by ML methods while dealing with complex temporal dependencies.

The authors developed a CNN-LSTM-SE model to overcome these limitations with innovations such as EEMD denoising for noise reduction, channel attention mechanism using the SE module for improved precision, and with the hybrid architecture the model captures both spatial and temporal features. When comparing the performance with traditional models, this hybrid architecture showed superior performance by achieving an accuracy of 98.5% and the precision, recall and F1 exceeded 97%.

2.4 Classification of Electrocardiogram Signals Based on Hybrid Deep Learning Models

This paper mainly helps us understand about the common issues in ECG arrhythmia classification and other similar ML tasks that is class imbalance. It occurs when there is a irregular data distribution between classes, where one class greatly outnumbers the other classes in a classification, thus causing an imbalance in the data. In the MIT-BIH dataset the number of beats belonging to normal beat (N) class is about 80% whereas the minority classes account for less than 20%. This imbalance results in overfitting of the majority class and also misclassification of the minority class beats.

This paper employs techniques such as Synthetic minority oversampling technique (SMOTE) for generating synthetic samples for the minority class. This method is used to balance the dataset without simply duplicating the existing data and thus reduces the risk of overfitting. It also employs under-sampling of the number of samples in the majority class to balance the dataset, but this method contains the risk of losing important information from the majority class. Without class balancing, the model might achieve higher accuracy but suffers greatly in precision or recall for minority classes. With SMOTE, the model showed higher precision, recall and F-score across all the classes. This paper effectively employs SMOTE to handle class imbalance and thus significantly improving the classification output.

2.5 Performance Study of Different Denoising Methods for ECG Signals

This paper helps us understand about the various noise sources from which ECG signals are corrupted, such as power line interference, baseline drift from respiration, muscle activity and poor electrode contact. ECG is susceptible to these types of noises which will distort the morphological features such as P-wave, QRS-complex, T-wave and U- wave and also the interval features of the ECG which leads to false diagnosis and improper treatment. Denoising of the signals is crucial while also preserving the signal's vital information for accurate medical diagnosis.

The paper discusses about some mainly used techniques for ECG denoising and also analyzes their performance. It examines methods such as wavelet transform, adaptive filtering, Savitzky-Golay filtering. The paper uses performance metrics such as PRD, SNR to assess the denoising method's effectiveness. The paper concludes by stating that at high noise levels, all the above mentioned methods show good results, but in mid-SNR ranges, RLS algorithm and Savitzky-Golay outperforms the wavelet methods. NeighBlock wavelet denoising methods performs the best by offering superior overall performance. Our proposed work uses wavelet decomposition and band-pass filtering methods for denoising.

2.6 ECG - based cardiac arrhythmias detection through ensemble learning and fusion of deep spatial–temporal and long-range dependency features

This paper was helpful in understanding the drawbacks with existing approaches that focus on individual deep learning models such CNN, LSTM and Transformers. CNN fails to capture long term temporal dependencies, LSTM struggle at very long range dependencies and Transformers tend to overfit and also have high computational demands when compared with other models. This paper proposes a hybrid deep learning framework by integrating CNN, CNN-LSTM and Transformer models for extracting spatial. Temporal and long range dependency features. This approach aims to fuse features from these models to create a comprehensive feature vector. It also uses a majority voting ensemble classifier combining SVM, Logistic Regression and Random Forest for the final classification. These traditional classifiers (SVM, LR, RF) are enriched with the features extracted by the DL models, thus results in improved performance.

This approach achieved an accuracy of 99.56% and high precision, recall and F-scores. This model outperforms other models using standalone classifiers and other ensemble approaches.

2.7 ECG-TransCovNet: A hybrid transformer model for accurate arrhythmia detection using Electrocardiogram signals

This paper states the limitations in traditional learning methods for ECG classification and proposes a novel ECG-TransCovNet hybrid architecture combining CNN and transformers. It also introduces a selective kernel (SK) module which dynamically adjusts receptive fields thus optimizing feature extraction. The paper also proposes a focal loss function to address the imbalance of data present in the dataset and the model is further optimized for reduced computational complexity. This ECG-TransCovNet approaches uses the strengths of CNN and Transformers to address the practical challenges in Arrhythmia detection. Our proposed work doesn't implement Transformers but rather implements a CNN-BiLSTM architecture.

Chapter 3

Methodology

3.1 Dataset

The MIT-BIH dataset developed by MIT and Beth Israel Hospital (BIH) is one of the most popular datasets for ECG arrhythmia classification. It contains 48 samples each recorded for an average of 30 minutes, obtained from 47 patients studied by the BIH arrhythmia Laboratory between 1975 and 1979. The dataset contains 2-channel ECG signals. The data was sampled at 360 Hz frequency with 11-bit resolution over a 10mV range. The ECG signals are annotated with beats labels by experts thus making it useful for training and testing arrhythmia detection systems.

Each recording consists of 2 channels of ECG data. In most of the signals, the upper signal is MLII and the lower signal is modified lead V1 and sometimes V2 or V5. The patients were 25 men in the age range of 32 to 89 years and 22 women in age range of 23 to 89 years. The dataset annotations include a total of 15 classes of beats. Our proposed work classifies the ECG signals into 5 classes as specified by the ANSI/AAMI57 standard.

ECG Signal classification standard specified by ANSI/AAMI EC57

Groups	ECG classes
Normal (N)	Normal(N) Left Bundle Brunch Block (L) Right Bundle Brunch Block (R) Atrial Escape (e) Nodal (junctional) escape (j)
Supraventricular ectopic beat (S)	Atrial Premature (A) Aberrant Atrial Premature (a) Nodal (Junctional) Premature (J) Supra-ventricular Premature (S)
Ventricular ectopic beat (V)	Premature Ventricular Contraction (V) Ventricular escape (E)
Fusion beat (F)	Fusion of Ventricular and Normal (F)
Unknown beat (Q)	Paced (/) Fusion of paced and Normal (f) Unclassifiable (Q)

3.2 Data Preprocessing

3.2.1 Denoising

ECG signals contain many kinds of noises and artifacts which can lead to wrong diagnosis and thus require denoising. By denoising the data, we improve the signal quality and thus enhancing the performance of the classification model. In this study we propose two denoising techniques – bandpass filtering and wavelet denoising are applied to the signals to filter out the unwanted noise while preserving the vital information present in the data.

Bandpass Filtering:

The ECG signal contains noises such as baseline wander and high frequency noises such as muscle artifacts or electrical interference. A bandpass filter is used to remove these types of noise. A custom bandpass filter is proposed with a cutoff frequency of 0.5 Hz which is used to smooth out the baseline drift by convolving the signal with a moving average filter and subtracting the baseline from the original signal. Then a high pass filter is proposed to attenuate the high frequency noise. The filters are applied iteratively to remove both high frequency and low frequency noise. This bandpass filter enhances the signal by isolating the frequencies in the range of 0.5-50 Hz and eliminates the unwanted components.

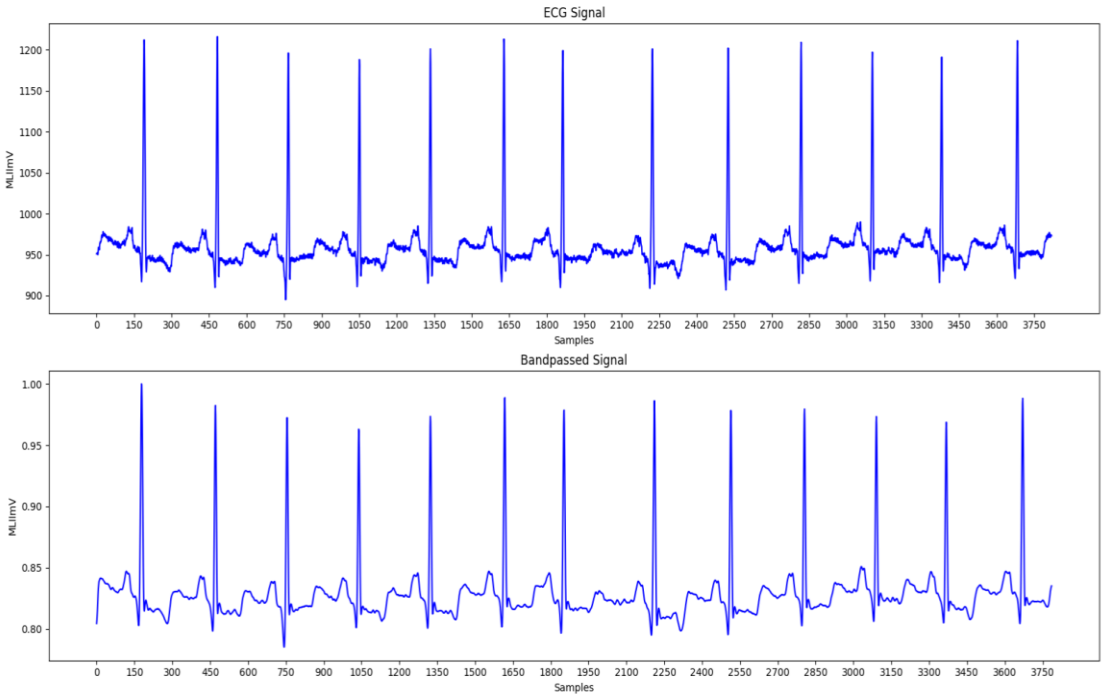

FIGURE 3.1: Plot of raw ECG signal and bandpassed signal

Discrete Wavelet Transform:

The wavelet transform is suitable technique for non-stationary signals like ECG. The denoising procedure using DWT involves 3 steps – Wavelet Decomposition, thresholding and Signal reconstruction. First the signal is decomposed into 9 components using the db5 wavelet. This splits the signal into 1 approximation and 9 detail coefficients. Here the detail coefficients represent the high frequency noise components. Now the thresholding technique is applied to the detail coefficients to reduce the noise. The threshold is determined using median absolute deviation method. The coefficients below this threshold value are set to zero and this removes the noise. Finally, the signal is reconstructed from the modified wavelet coefficients using Inverse Discrete Wavelet Transform (IDWT). At the end, the denoise version of the EC G signal is obtained.

3.2.2 R-Peak detection and segmentation

R-Peak Detection:
R-Peak detection is crucial in ECG signal processing. In this study we propose the Pan-Tompkins algorithm for QRS complex detection. According to the algorithm, the ECG signal is filtered using a bandpass filter to remove the high frequency noise and then a derivative of the signal is computed to emphasize the slopes of the QRS complex. It helps identify the rapid changes that occur at the R-Peak of the ECG signal. Then the differentiated signal is squared to amplify the R-wave so that the R-peaks become more distinct. Finally moving window integration is applied to smooth the signal and make the R-peaks more prominent. It highlights the areas where the signal is strong which makes it easier to identify the location of R-Peaks.

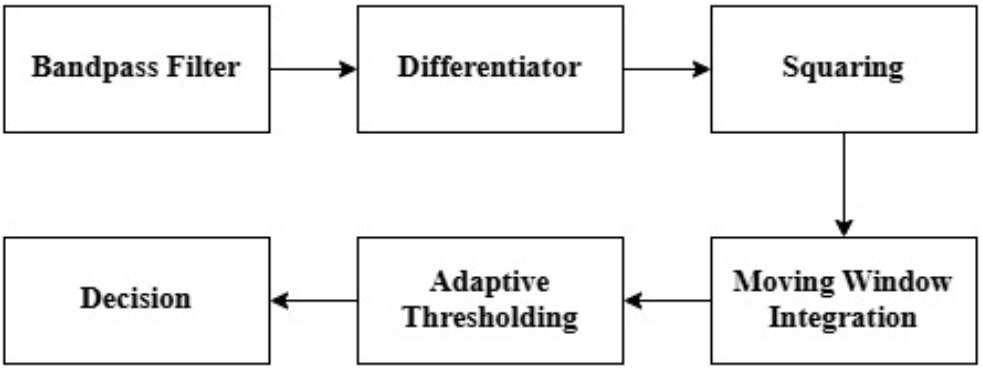

FIGURE 3.2: Block Diagram of the Pan-Tompkins algorithm

Next to detect the R-Peaks, the slope of the signal is calculated using convolution, which helps to identify the approximate R-Peaks. The RR-intervals are adjusted based on the calculated approximate R-Peaks. A dynamic threshold is adjusted based on the RR intervals. This helps in finding missed or false peaks. A searchback method is applied to find the exact location of the peak around each approximate peak. T-waves are also detected to distinguish them from the R-peaks as t-waves can be mistaken as R-Peaks. Finally, the refined R-Peaks are determined which is later used for segmentation of the signals.

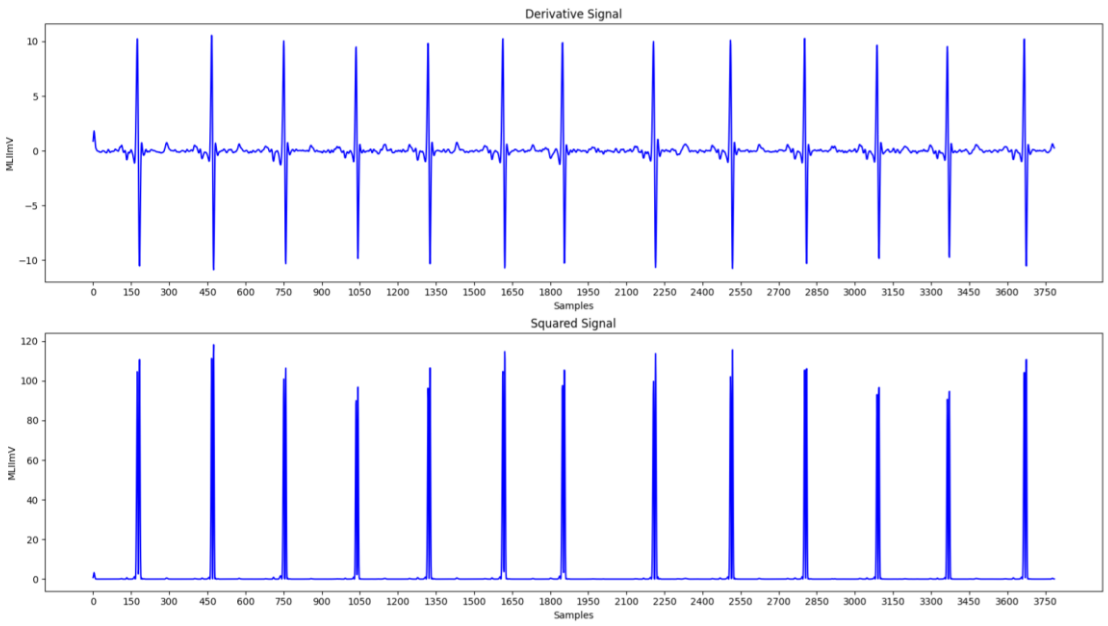

FIGURE 3.3: Plot of differentiated and squared ECG signal

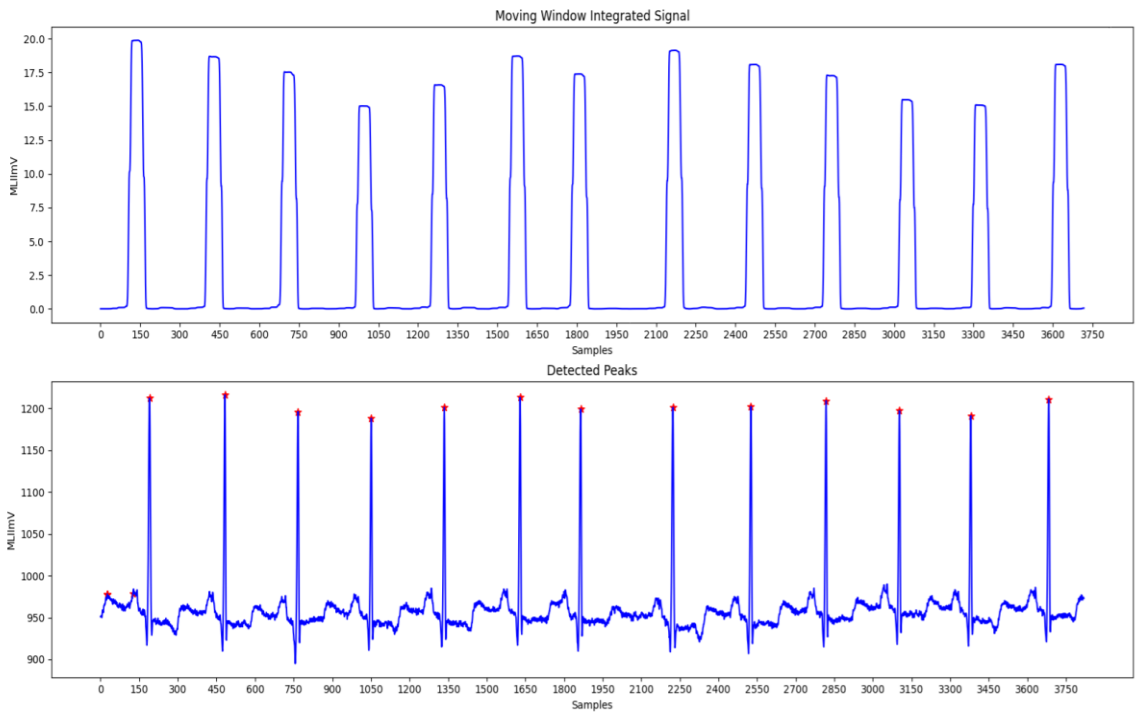

FIGURE 3.4: Plot of moving windows integrated signal and R-peaks annotated signal.

Segmentation:
We perform segmentation to divide the continuous ECG signal into smaller, fixed length windows as deep learning models require input data of consistent size. Even though we detected the R-Peaks of the signals by detecting the QRS complex in the previous step, the detected R-peaks had some false positives and negatives when compared with the R-peak locations annotated in the dataset which were labelled by experts. So, to avoid incorrect segmentation and classification, we used the annotated R-peaks from the dataset for segmentation in our proposed work. We extracted the R-peaks from the annotation file with each record.

The signal is segmented with fixed length windows around each R-peak where the windows are centered at the peak with fixed size of samples. Each segment contains a complete QRS complex and the segments are labelled based on the annotation at the corresponding R-Peak. The segments are labelled using the 5 classes N, S, V, F, Q. Later the labels ate mapped to numeric form for the training purpose.

Class 0	Normal beats (N)
Class 1	Supraventricular beats (S)
Class 2	Ventricular beats (V)
Class 3	Fusion beats (F)
Class 4	Unknown beats (Q)

FIGURE 3.5: Table showing the mapping of class labels to numbers

3.2.3 Handling Class Imbalance and Data Splitting

Data Splitting:
The proposed model works with both the channels of ECG data from the dataset for training the classification model. First the ECG data is split into 2 parts each containing one channel of data and then stacked into a 3D array. Each signal has 300 data points. The data from the channels are stacked along the last axis to form a 3D array of shape (samples, 300, 2). Thus, each ECG sample has 300 time points with 2 features consisting of the 2 channels.

The dataset containing the data of both the channels is split into training and testing sets. 20% of the data is reserved for testing and the remaining 80% of the data is used to train the classification model.

Handling Class Imbalance:
In the MIT-BIH dataset, the normal beats (N) dominate the dataset as normal beats make up a large portion of the dataset thus making it the majority class. Other classes such as S, V, F, Q which represent abnormal beats have very few samples when compared to N. This imbalance can affect the classification result as the model tends to bias their predictions towards the majority class. This can lead to false negatives in the classification. This class imbalance can lead to overfitting and thus the model will perform well in predicting the majority class but fails while predicting the minority class.

To address the class imbalance in the dataset, we apply random sampling to the majority class to reduce the over representation of the normal beats (N) by randomly selecting 50000 samples from it. This ensure that the model does that get biased towards the minority classes by learning too much about the N class. After handling class 0, we balance the minority classes (1,2,3,4) by oversampling these minority classes. We apply Synthetic Minority Over-sampling technique to generate synthetic samples instead of just simply duplicating the existing samples. Each class (1,2,3,4) are oversampled to 50000 samples to match the number of samples in the majority class. After resampling the majority class and oversampling the minority class separately, both the resampled datasets are combined into a single dataset and randomly shuffled so that the data is not ordered which could introduce bias.

Class	**Total Samples**	**Testing set**	**Training set**	**Training set (balanced)**
N	90472	18095	72377	50000
S	2775	555	2220	50000
V	7229	1446	5783	50000
F	802	161	641	50000
Q	8027	1606	6421	50000

FIGURE 3.6: Table displaying the number of samples in each class

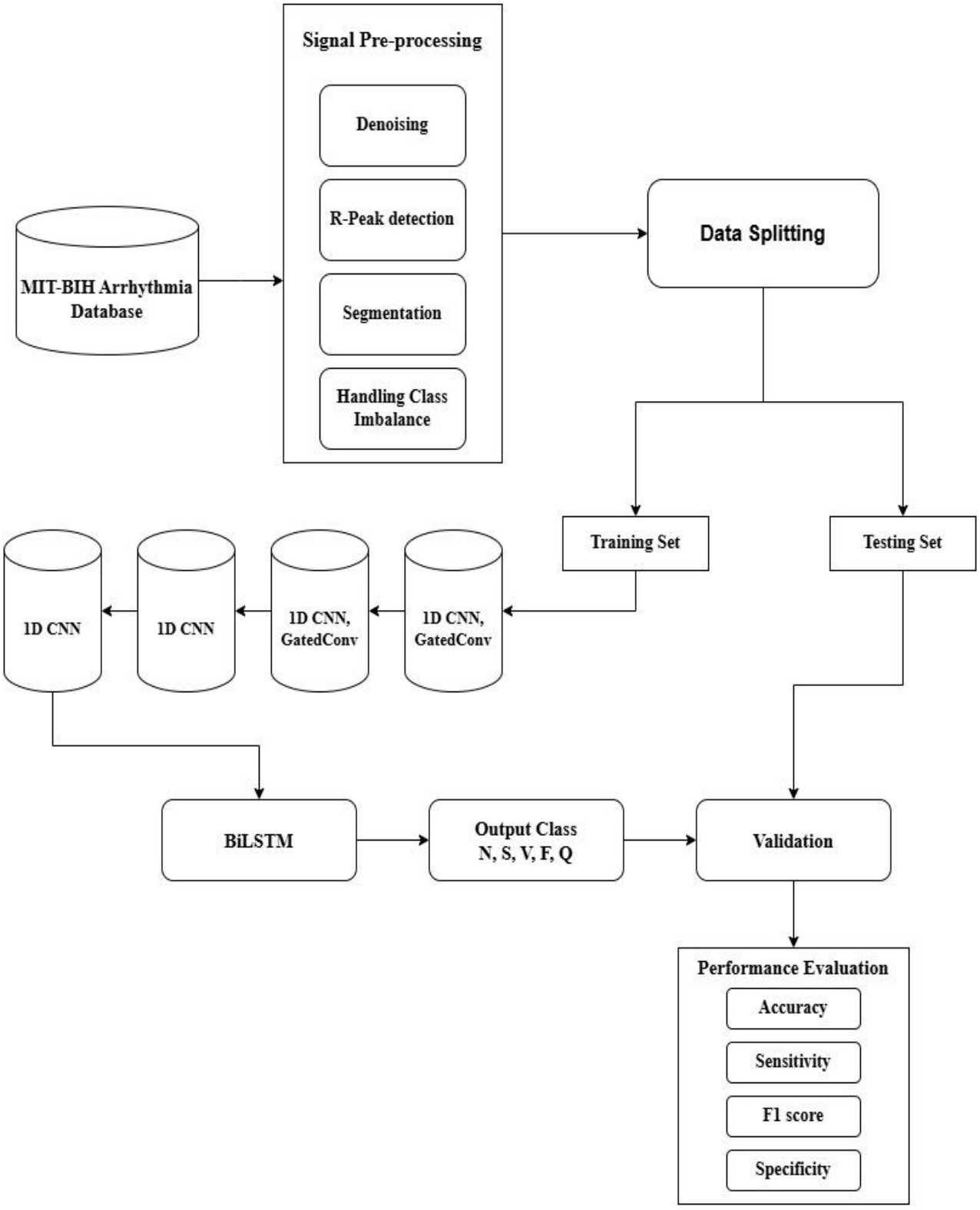

FIGURE 3.7: Block Diagram of the proposed model.

3.3 Model Architecture

The core of our proposed work is to develop a hybrid deep learning model by combining CNN, BiLSTM and gated convolution networks to classify ECG signals into 5 primary heartbeat classes. Convolutional Neural networks are a powerful architecture while dealing with temporal and spatial data such as ECG signals. Deep CNN are very effective at learning complex relationships between the input and output. Our proposed model uses 4 CNN layers in combination with Gated Convolution Layers which is introduces to enhance feature representation using self-excitation. Pooling layers are used for dimensionality reducing thus reducing computational complexity while preserving the important features. The CNN layers are combined with Bidirectional LSTM layers to capture temporal dependencies in ECG data. These are helpful due to sequential nature of ECG data.

3.3.1 Proposed model architecture:

1) **Input Layer:** This layer takes an input sequence of shape (samples, 300, 2) where 300 represents the length of the ECG signal and 2 represents the 2 channels of the ECG signals.
2) **Convolution Block 1**
   - i. **Conv1D layer:** The first CNN layer with 16 filters performs standard convolution on the input and produces 16 feature maps. Each filter learns a different feature from the input. The filter has a length of 21 and the filter moves 1 step at a time due to stride=1.Padding is set to same and the Rectified Linear Unit (ReLu) activation function is applied after the convolution. This introduces non linearity and helps the model to learn more complex features
   - ii. **Batch Normalization:** To normalize the outputs of the previous layer.
   - iii. **Gated Convolution Layer:** A custom layer to apply the gated convolution operation. It has 16 filters with kernel length of 21. This custom layer performs a convolution on the input data and uses two additional convolutional layers gate1 and gate2 with sigmoid activations. This helps the model to focus on the important features. The output is multiplied by 2 gates. This is where the gating mechanism comes into play.
   - iv. **Max Pooling Layer:** This layer is used to down sample the data to reduce the dimensionality when retaining important features.
3) **Convolution Block 2**
   - i. **Conv1D layer:** Another CNN layer similar to 1st block but uses 32 filters with kernel size of 23 and strides =1 with same padding and relu activation.
   - ii. **Batch Normalization:** Batch normalization applied again to normalize the output.
   - iii. **Gated Conv Layer:** Another instance of the gated convolution layer with 32 filters and a kernel size of 23 to match with the CNN layer and again adds a gating mechanism to the convolution operation.
   - iv. **Max Pooling layer:** Used again to down sample the feature maps.
4) **Convolution Block 3**
   - i. This block again works similarly to the previous blocks with a 1D CNN layer with 64 filters and kernel size of 25 and relu activation function. Due to larger kernel size the model captures broader temporal relationships in the ECG data.
   - ii. This block again has a batch normalization and layer and also average pooling layer to perform down sampling. Unlike max pooling, avg pooling computes average value within a window and thus represents the input more smoothly and reduces the risk of overfitting.
5) **Convolution Block 4:** This block has the final 1D CNN layer which uses 128 filters and a kernel size of 27 with stride=1 and same padding and also relu activation function. This block also has a batch normalization layer like the other 3 blocks.
6) **Bidirectional LSTM layers:**
   - i. **First BiLSTM layer:** It is a type of recurrent layer this is used to capture dependencies in both directions. It processes input in 2 ways - forward pass and backward pass. This layer helps the model learn about temporal dependencies from

sequential data like ECG. The LSTM has 64 units and has return_sequence = True.

ii. **Dropout Layer:** This layer is used to randomly drop out certain fraction of the units during training to prevent overfitting. It has rate=0.2 which means 20% of the units will be dropped.

iii. **Second BiLSTM layer:** Another BiLSTM layer with fewer units. It has 32 units and return_sequence=True. By stacking another BiLSTM layer the model captures even more complex temporal relationships.

7) **Fully connected and output layers:** The final layers consist of flatten layer to reshape the output from the BiLSTM layer into a 1D vector to feed it to the Dense layer. Then the Dense layer with 128 units is fully connected takes with flattened output and also uses a relu activation function. Another dropout layer with a dropout rate of 30% and further prevents overfitting. Then the final Dense layer which is the output layer is fully connected with 5 units which corresponds to the 5 classes of the ECG signals. Softmax activation function is used to get a probability distribution across the 5 classes as output. The model will predict the class with the highest probability. This layer provides the final output for the classification.

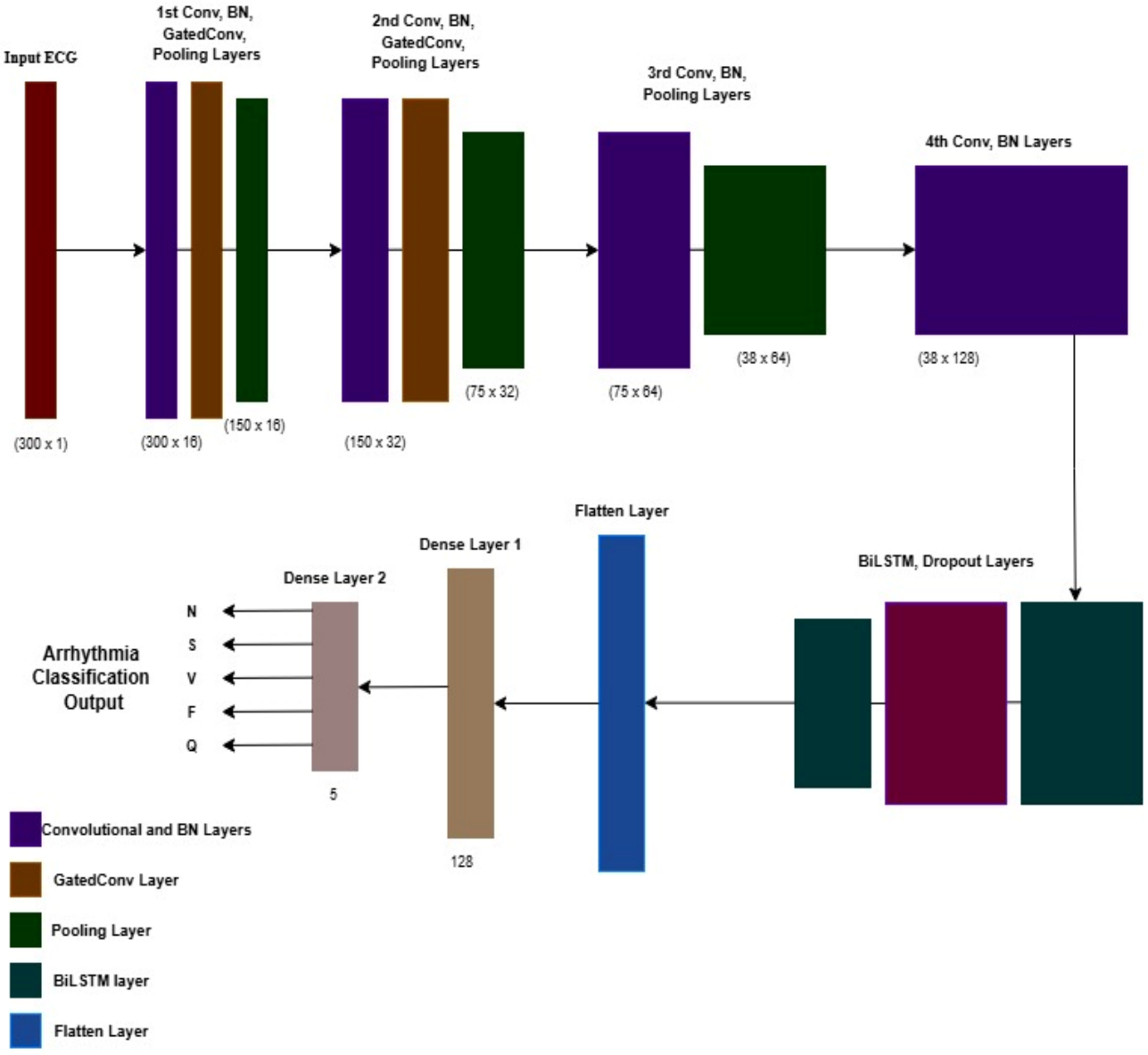

FIGURE 3.8: Architecture of the proposed model.

3.3.2 Gated Convolution:

In our proposed work, Gated Convolution is a type of convolutional layer which was designed to control which features are allowed to pass through the network. This layer introduces a gating mechanism in the model which helps the model to focus on the most important parts of the input while suppressing the less important information. This is particularly helpful in time-series data like ECG.

Architecture of the gated convolution layer:

i. **Input Layer:** The Gated Convolution layer consists of a input layer which takes the ECG signal as input.

ii. **Conv1 layer:** The first convolution layer conv1 is the first step in the gated convolution where it extracts features of the input signal. This conv1 layer same padding and a dilation rate which control the spacing between kernel elements:

iii. **Gates:** introducing gates in the model is the core idea of gated convolution. These gates are learned filters that modulates the output of the conv1 layer. There are 2 gates in our architecture. Sigmoid activation function is applied to these gates

iv. **Gated Output:** This is generated by the element wise multiplication between the output of the main convolution and the 2 gates.

v. **Batch Normalization:** This is applied to stabilize and accelerate training by normalizing the activation of the gated output.

vi. **Skip connection:** A skip connection is added to the model to allow the flow of gradients and prevent information loss in deeper layers

vii. **Output layer:** The final output of this custom layer is the sum of gated output and skip connection. The output finally passes through an activation function.

Layer	Filters	Kernel Size
Input Layer	-	-
Conv1D	16	21 x 1
BatchNorm	-	-
GatedConv	16	21 x 1
MaxPooling1D	-	3 x 1
Conv1D	32	23 x 1
BatchNorm	-	-
GatedConv	32	23 x 1
MaxPooling1D	-	3 x 1
Conv1D	64	24 x 1
BatchNorm	-	-
AveragePooling1D	-	3 x 1
Conv1D	128	27 x 1
BatchNorm	-	-
BiLSTM	-	-
Dropout	-	-
BiLSTM	-	-
Flatten	-	-
Dense	-	-
Dropout	-	-
Dense	-	-

FIGURE 3.9: Summary of the proposed model.

3.4 Model Training

The CNN-BiLSTM model is trained using the Adam optimizer for adaptive learning rate adjustments. The sparse categorical crossentropy loss function was used and accuracy was used as the performance metric to monitor the model during the training. The ECG dataset divided into training and validation sets. The model was trained for 30 epochs and with a batch size of 128. 20% of the training data was reserved for validation. Metrics such as training accuracy and loss were visualized using accuracy curve and loss curve.

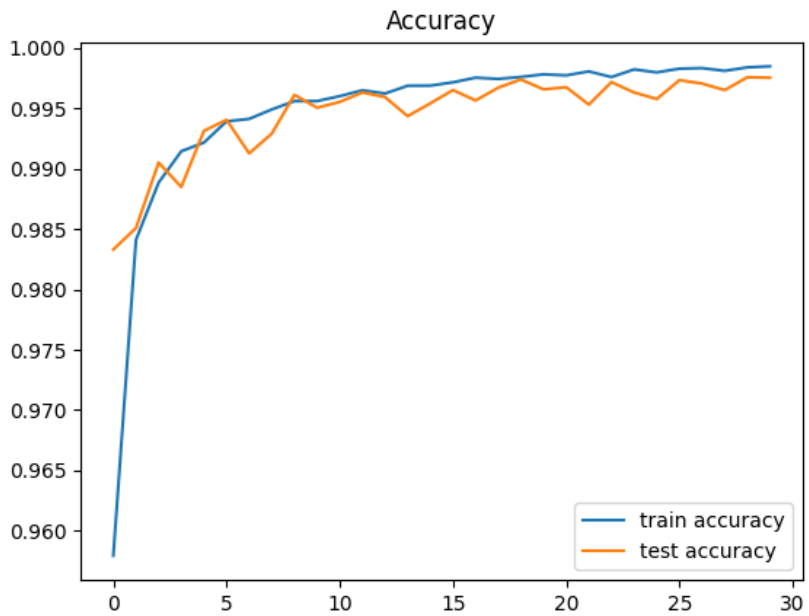

FIGURE 3.10: Accuracy curve.

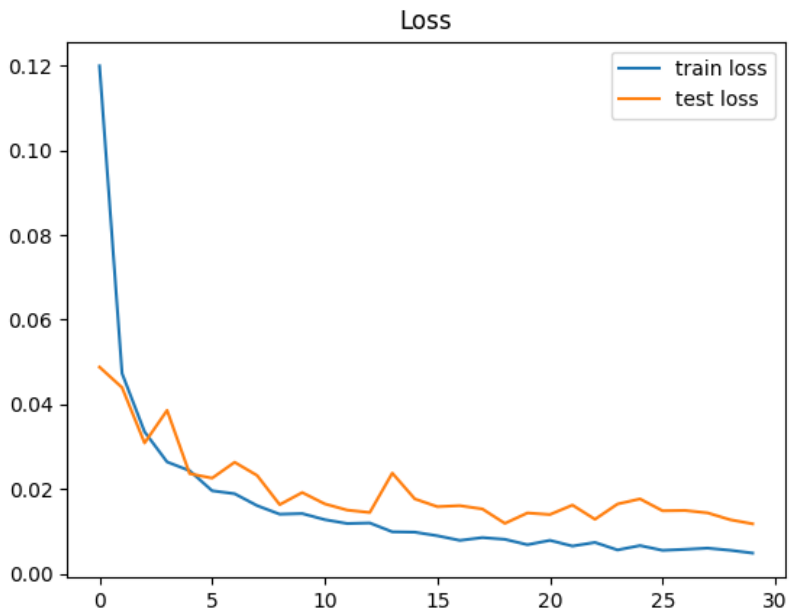

FIGURE 3.11: Loss curve.

Chapter 4

Results and Discussions

Objective

Our primary objective during this study was to design and implement a deep learning model which was capable of classifying ECG signals as normal or abnormal beats. With the yearly increase in deaths caused due to cardiovascular diseases and growing prevalence of heart conditions, the need for automated cardiac monitoring systems has become very crucial. Our project aimed to develop a hybrid classification model that could effectively detect abnormal ECG signal at a higher accuracy in comparison to traditional machine learning models and standalone deep learning models. We wanted our model to handle the complex time-series data effectively and extract important features essential for classifying different arrhythmia classes.

In our study, several challenges were addressed. Imbalanced data which is a common issue in medical datasets was addressed using SMOTE to prevent the model from overfitting and affect the performance of the detection model. We wanted to balance the model's computational complexity without compromising its performance, so that it can demonstrate good performance even in resource constrained environments. We aimed to build a model that can capture both spatial and temporal features present in the input to overcome the limitations of other models when processing ECG signals.

4.1 Results and comparison

The performance of the proposed model was evaluated and compared with another configuration of the model. The other model included 4 standard CNN layers similar to the proposed model followed by unidirectional LSTM layers instead of BiLSTM layers and also no gating mechanism was introduced. The filters and kernel lengths of the secondary model were matched with the proposed model.

Both the model's performances were evaluated on both the training and validation set. To evaluate the classification model, and to assess the model's performance we used metrics such as accuracy, loss, sensitivity, specificity, F1 score and per class accuracy and confusion matrix. Confusion matrix was used to analyze the distribution of the predictions. It provided a detailed breakdown of the correct and incorrect classifications.

Formulas used to calculate the model validation metrics are shown below.

1) Sensitivity / Recall $= \dfrac{TP}{TP + FN}$

2) Specificity $= \dfrac{TN}{TN + FP}$

3) Accuracy $= \dfrac{TP + TN}{TP + TN + FP + FN}$

4) Precision $= \dfrac{TP}{TP + FP}$

5) F1 score $= 2 \times \dfrac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

FIGURE 4.1: Formulas to calculate validation metrics.

The proposed model achieved a training accuracy of 99.81% and a validation accuracy of 98.88% with a training loss of 0.66 and validation loss of 7.90.

```
Training Accuracy: 99.81, Test Accuracy: 98.88
Training Loss: 0.66, Test Loss: 7.90
```

The CNN-LSTM model achieved a training accuracy of 98.72% and a validation accuracy of 97.86% with a training loss of 5.84 and validation loss of 6.84.

```
Training Accuracy: 98.72, Test Accuracy: 97.86
Training Loss: 5.84, Test Loss: 6.84
```

Thus, from the obtained results it is clear that the CNN-BiLSTM model with gated convolutions displayed better performance in predicting both the training set and also the validation set. From this we infer that the architecture of our proposed model performs better than the CNN-LSTM model.

Model	**Training Accuracy**	**Test accuracy**	**Train loss**	**Test loss**
CNN-BiLSTM with gated convolution	99.81	98.88	0.66	7.90
CNN-LSTM	98.72	97.86	5.84	6.84

FIGURE 4.2: Table displaying the accuracy results of the models

4.1.1 Confusion Matrix

The confusion matrix of the proposed model showed well separated true and predicted labels but showed some confusion in classes with fewer samples such as class F and Q.

Confusion matrix of proposed model:

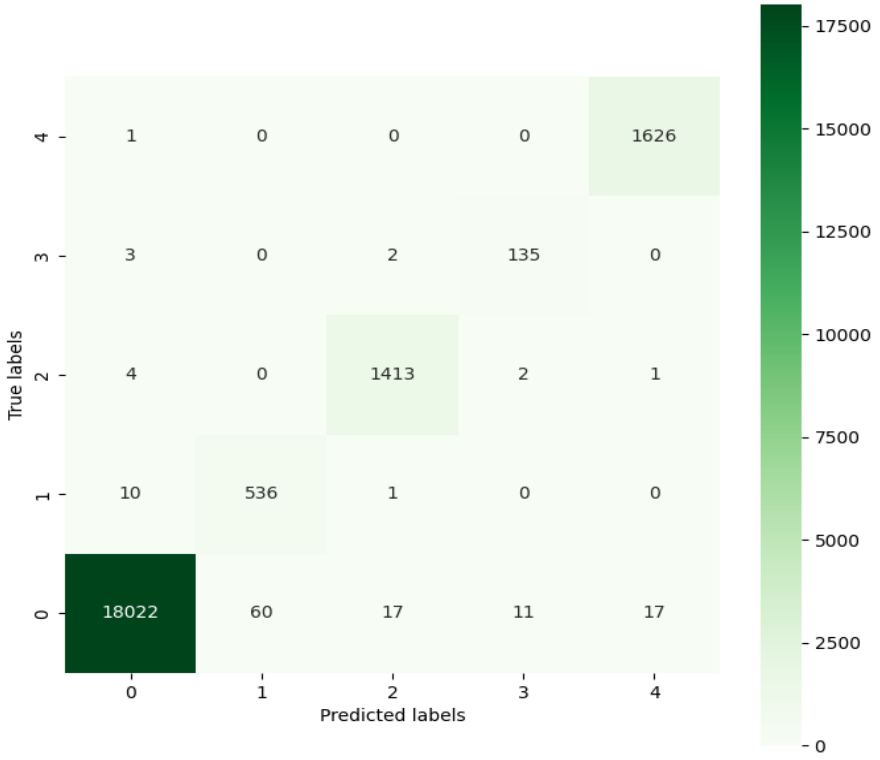

FIGURE 4.3: Confusion Matrix of the proposed model

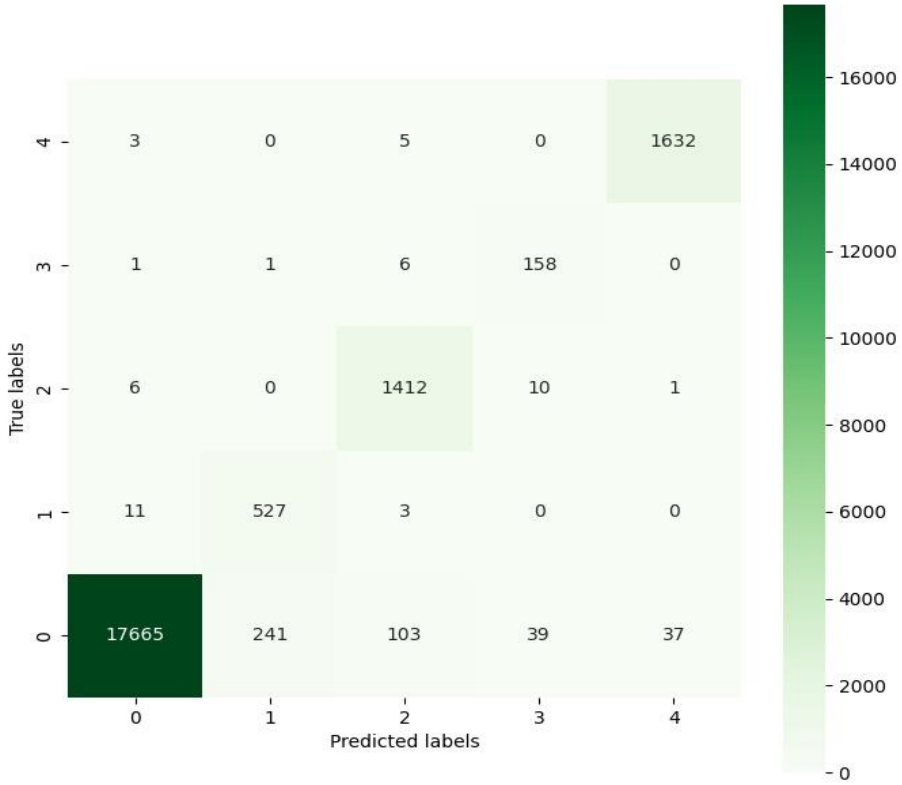

FIGURE 4.4: Confusion Matrix of the CNN-LSTM model

4.1.2 Per-class score:

The per class accuracy matrices provided us the model's performance in predicting each class type. The model performed well in predicting common classes such as N and V but showed slightly lower accuracy and F1 score while classifying minority classes such as F class due to its small sample availability.

The model achieved a per class sensitivity of 99.186 (N), 93.001 (S), 97.639 (V), 86.49 (F) and 99.76 (Q) and a F1 score of 99.398 (N), 88.75 (S), 97.43 (V), 85.10(F) and 99.35 (Q).

	N	S	V	F	Q
Sensitivity (%)	99.186272	93.001842	97.639485	86.419753	99.763733
Specificity (%)	98.155954	99.577822	99.809412	99.875570	99.910750
F1 Score (%)	99.398108	88.752197	97.430407	85.106383	99.352941
Per Class Accuracy (%)	99.007365	99.414482	99.670646	99.775857	99.899364

FIGURE 4.5: Per-class score of the model

As expected, when dealing with the minority classes such as S and F, the model struggled in classifying beats belonging to those classes. This resulted in reduced sensitivity and F1 score for those classes. The model showed only 93% sensitivity in S class and 86.419% in F class. The model got a F1 score of only 88.75% in S class and 85.10% in F class.

The model struggles to classify these classes effectively even after implementing class balancing techniques such as SMOTE because even after class balancing, the model's bias towards majority classes is not fully addressed. The synthetic data generated doesn't fully capture the complex patterns of real F class beats and thus leads to poor generalization.

4.1.2 Performance comparison after class balancing

The imbalanced dataset was balanced using SMOTE which created a significant impact on the performance of the model in classifying beats belonging to the minority class. We compared the proposed model's performance trained after class balancing with the model trained without handling the class imbalance.

Model	**F class Accuracy**	**F1 Score**
CNN-LSTM model with class balancing	99.77	85.1
CNN-LSTM model without class balancing	99.60	78.41

FIGURE 4.6: Model performance comparison after SMOTE

Performance score before class balancing:

	N	S	V	F	Q
Sensitivity (%)	98.725798	93.035714	95.862553	91.329480	99.618563
Specificity (%)	97.990354	99.446035	99.667238	99.668019	99.916207
F1 Score (%)	99.152402	86.905755	95.560993	78.411911	99.271460
Per Class Accuracy (%)	98.600247	99.281826	99.419057	99.602031	99.894790

Performance score after class balancing:

	N	S	V	F	Q
Sensitivity (%)	99.186272	93.001842	97.639485	86.419753	99.763733
Specificity (%)	98.155954	99.577822	99.809412	99.875570	99.910750
F1 Score (%)	99.398108	88.752197	97.430407	85.106383	99.352941
Per Class Accuracy (%)	99.007365	99.414482	99.670646	99.775857	99.899364

There is a significant drop in both the accuracy and F1 score while predicting the minority class F without handling the class imbalance. These scores show the impact of SMOTE technique used in the proposed model for handling the imbalanced MIT-BIH dataset.

Chapter 5

Conclusion and Future Scope

This study demonstrates the effectiveness of a hybrid deep learning architecture in classification tasks such as ECG arrhythmia detection. By combining Convolutional Neural Networks and Recurrent Neural Networks such as LSTM, the model was efficient in extracting spatial or local features, temporal or long-term dependencies from ECG signals and achieved high classification accuracy. By using SMOTE, we handled class imbalances present in MIT-BIH dataset which prevented the model from overfitting and improved the accuracy and F score in classifying minority classes such as S and F classes.

This study highlights the importance of using deep learning architectures for the accurate detection of arrhythmia. This study shows the potential of this approach to support clinical decision making and aid medical practitioners in early and accurate diagnosis of CVDs.

Future scope:

While our proposed model displays promising results, the system can be improved in several avenues in the future. By implementing more complex architectures such as Transformer architecture, graph neural networks, transfer learning techniques, supervised learning techniques, multimodal learning could further enhance our model. Also, by using multi lead ECG data instead of 2-lead data, we can extract more important features from the signals which could further improve the classification performance of our model.

By addressing these areas, our model can be evolved into a more robust and scalable solution for automated diagnosis of cardiovascular diseases using deep learning techniques.

Chapter 6

Appendix

Preprocessing module:

DWT Function:

```
def dwt(data):
    cf = pywt.wavedec(data=data, wavelet='db5', level=9)
    a9, d9, d8, d7, d6, d5, d4, d3, d2, d1 = coef

    t = (np.median(np.abs(d1)) / 0.6745) * (np.sqrt(2 * np.log(len(d1))))
    d1.fill(0)
    d2.fill(0)
    for i in range(1, len(coef) - 2):
        cf[i] = pywt.threshold(coef[i], t)

    denoised_data = pywt.waverec(coeffs=coef, wavelet='db5')
    return denoised_data

ann = wfdb.rdann(root + '100', 'atr')
R_peak = ann.sample
R_class = ann.symbol
```

R-Peak detection:

```
class preprocess():
    def bp_filter(self, signal):
        result = None
        sig = signal.copy()

        sampling_rate = 360
        cutoff_freq = 0.5
        window_size = int(sampling_rate / cutoff_freq)

        baseline = np.convolve(sig, np.ones(window_size) / window_size, mode='same')

        sig = sig - baseline

        for index in range(len(signal)):
            sig[index] = signal[index]
```

```
        if (index >= 1):
            sig[index] += 2 * sig[index - 1]

        if (index >= 2):
            sig[index] -= sig[index - 2]

        if (index >= 6):
            sig[index] -= 2 * signal[index - 6]

        if (index >= 12):
            sig[index] += signal[index - 12]

    result = sig.copy()

    for index in range(len(signal)):
        result[index] = -1 * sig[index]

        if (index >= 1):
            result[index] -= result[index - 1]

        if (index >= 16):
            result[index] += 32 * sig[index - 16]

        if (index >= 32):
            result[index] += sig[index - 32]

    max_val = max(max(result), -min(result))
    result = result / max_val
    return result

def derivative(self, signal):
    result = signal.copy()

    for index in range(len(signal)):
        result[index] = 0

        if (index >= 1):
            result[index] -= 2 * signal[index - 1]

        if (index >= 2):
            result[index] -= signal[index - 2]

        if (index >= 2 and index <= len(signal) - 2):
            result[index] += 2 * signal[index + 1]

        if (index >= 2 and index <= len(signal) - 3):
            result[index] += signal[index + 2]

        result[index] = (result[index] * annotation.fs) / 8
    return result
```

```
def squaring(self, signal):
    result = signal.copy()

    for index in range(len(signal)):
        result[index] = signal[index] ** 2
    return result

def mw(self, signal):

    result = signal.copy()
    win_size = round(0.150 * annotation.fs)
    sum = 0

    for j in range(win_size):
        sum += signal[j] / win_size
        result[j] = sum

    for index in range(win_size, len(signal)):
        sum += signal[index] / win_size
        sum -= signal[index - win_size] / win_size
        result[index] = sum
    return result

def approx_peak(self):

    slopes = sg.fftconvolve(self.m_win, np.full((25,), 1) / 25, mode='same')

    for i in range(round(0.5 * self.sf) + 1, len(slopes) - 1):
        if (slopes[i] > slopes[i - 1]) and (slopes[i + 1] < slopes[i]):
            self.peaks.append(i)

def adj_rr(self, ind):

    self.RR1 = np.diff(self.peaks[max(0, ind - 8): ind + 1]) / self.sf

    self.avg1= np.mean(self.RR1)
    abg2= self.avg1

    if (ind >= 8):
        for i in range(0, 8):
            if (self.low < self.RR1[i] < self.high):
                self.RR2.append(self.RR1[i])

                if (len(self.RR2) > 8):
                    self.RR2.remove(self.RR2[0])
                    RR_Average2 = np.mean(self.RR2)

    if (len(self.RR2) > 7 or ind < 8):
        self.low = 0.92 * avg2
        self.high= 1.16 * avg2
        self.missed = 1.66 * svg2
```

```
def r-peaks(self):

    self.approx_peak()

    for ind in range(len(self.peaks)):

        peak_val = self.peaks[ind]
        win_300ms = np.arange(max(0, self.peaks[ind] - self.win_150ms),
                              min(self.peaks[ind] + self.win_150ms, len(self.b_pass) - 1), 1)
        max_val = max(self.b_pass[win_300ms], default=0)

        if (max_val != 0):
            x_coord = np.asarray(self.b_pass == max_val).nonzero()
            self.probable_peaks.append(x_coord[0][0])

        if (ind < len(self.probable_peaks) and ind != 0):

            self.adjust_rr_interval(ind)

            if (self.avg1 < self.low or self.avg1 > self.missed):
                self.th1 /= 2
                self.thf1 /= 2

            RRn = self.RR1[-1]

            self.searchback(peak, RRn, round(RRn * self.samp_freq))

            self.find_t_wave(peak, RRn, ind, ind - 1)

        else:
            self.adjust_th(peak, ind)

        self.update_th()
    self.ecg_sb()
    return self.result
```

Handling Data imbalance using SMOTE:

```
Y_reduced = LabelEncoder().fit_transform(Y_reduced)

ecg_data_channel1 = ecg[:, :300]
ecg_data_channel2 = ecg[:, 300:600]

ecg_data = np.stack((ecg_data_channel1, ecg_data_channel2), axis=-1)
ecg_data_flat = ecg_data.reshape(ecg_data.shape[0], -1)

ecg_lable = Y_reduced.reshape(-1, 1)

ecg_5_class = np.hstack((ecg_data_flat, ecg_lable))

from sklearn.model_selection import train_test_split
```

```
ecg_data = ecg_5_class[:, :600].reshape(-1, 300, 2)
ecg_label = ecg_5_class[:, 600]
x_train, x_test, y_train, y_test = train_test_split(ecg_data, ecg_label,
                                                    random_state=104,
                                                    test_size=0.20,
                                                    shuffle=True)

ytr= y_train.reshape(-1, 1)
yte = y_test.reshape(-1, 1)

train_data = np.hstack((x_train.reshape(x_train.shape[0], -1), ytr))
test_data = np.hstack((x_test.reshape(x_test.shape[0], -1), yte))

train = pd.DataFrame(train_data)

class_0 = train.loc[(train[600] == 0)]
class_1234 = train.loc[(train[600] != 0)]

from sklearn.utils import resample

train_rs = class_0.sample(n=50000, random_state=42)

train_rs = train_rs.to_numpy()

train_1234 = class_1234.to_numpy()
X_4_class, Y_4_class = train_1234[:, :-1], train_1234[:, -1]

from imblearn.over_sampling import SMOTE

strat = {1: 50000, 2: 50000, 3: 50000, 4: 50000}
oversample = SMOTE(sampling_strategy=strat)
X, Y = oversample.fit_resample(X_4_class, Y_4_class)

Y = Y.reshape(-1, 1)
train_1234_rs = np.hstack((X, Y))

train_data_rs = np.vstack((train_rs, train_1234_rs))

np.take(train_data_rs, np.random.permutation(train_data_rs.shape[0]), axis=0, out=train_data_rs)

x_train = train_data_rs[:, :-1].reshape(-1, 300, 2)
y_train = train_data_rs[:, -1]

return x_train, x_test, y_train, y_test
```

Main Model Building:

```
class GatedConv(tf.keras.layers.Layer):
    def __init__(self, filters, kernel_size, dilation_rate=1, activation='relu'):
        super(GatedConv, self).__init__()
        self.filters = filters
        self.kernel_size = kernel_size
        self.dilation_rate = dilation_rate
```

```
        self.activation = activation

        def build(self, input_shape):
            self.conv1 = tf.keras.layers.Conv1D(self.filters, self.kernel_size,
                                padding='same', dilation_rate=self.dilation_rate)
            self.gate1 = tf.keras.layers.Conv1D(self.filters, self.kernel_size,
                                padding='same', activation='sigmoid', dilation_rate=self.dilation_rate)
            self.gate2 = tf.keras.layers.Conv1D(self.filters, self.kernel_size,
                                padding='same', activation='sigmoid', dilation_rate=self.dilation_rate)
            self.skip_connection = tf.keras.layers.Conv1D(self.filters, 1, padding='same')
            self.bn = tf.keras.layers.BatchNormalization()
            if self.activation == 'relu':
                self.activation_fn = tf.keras.layers.ReLU()
            elif self.activation == 'leakyrelu':
                self.activation_fn = tf.keras.layers.LeakyReLU(alpha=0.2)
            else:
                self.activation_fn = None

        def call(self, inputs):
            conv_out = self.conv1(inputs)
            gate1_out = self.gate1(inputs)
            gate2_out = self.gate2(inputs)
            gated_out = tf.keras.layers.Multiply()([conv_out, gate1_out, gate2_out])
            gated_out = self.bn(gated_out)
            skip_out = self.skip_connection(inputs)
            output = tf.keras.layers.Add()([gated_out, skip_out])

            if self.activation_fn is not None:
                output = self.activation_fn(output)

            return output

def build_model():
    model_1 = tf.keras.models.Sequential([
        tf.keras.layers.InputLayer(input_shape=(300, 2)),
        tf.keras.layers.Conv1D(filters=16, kernel_size=21, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.BatchNormalization(),
        GatedConv(filters=16, kernel_size=21),
        tf.keras.layers.MaxPool1D(pool_size=3, strides=2, padding='SAME'),

        tf.keras.layers.Conv1D(filters=32, kernel_size=23, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.BatchNormalization(),
        GatedConv(filters=32, kernel_size=23),
        tf.keras.layers.MaxPool1D(pool_size=3, strides=2, padding='SAME'),

        tf.keras.layers.Conv1D(filters=64, kernel_size=25, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.AvgPool1D(pool_size=3, strides=2, padding='SAME'),

        tf.keras.layers.Conv1D(filters=128, kernel_size=27, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.BatchNormalization(),
```

```
        tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32, return_sequences=True)),

        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dropout(rate=0.3),
        tf.keras.layers.Dense(5, activation='softmax')
    ])
    return model_1
```

CNN-LSTM model for performance comparison:

```
def build_model2():
    model_2 = tf.keras.models.Sequential([
        tf.keras.layers.InputLayer(input_shape=(300, 2)),
        tf.keras.layers.Conv1D(filters=16, kernel_size=21, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.MaxPool1D(pool_size=3, strides=2, padding='SAME'),
        tf.keras.layers.Conv1D(filters=32, kernel_size=23, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.MaxPool1D(pool_size=3, strides=2, padding='SAME'),
        tf.keras.layers.Conv1D(filters=64, kernel_size=25, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.AvgPool1D(pool_size=3, strides=2, padding='SAME'),
        tf.keras.layers.Conv1D(filters=128, kernel_size=27, strides=1, padding='SAME', activation='relu'),
        tf.keras.layers.LSTM(128, return_sequences=True),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.LSTM(64, return_sequences=True),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dropout(rate=0.2),
        tf.keras.layers.Dense(5, activation='softmax')
    ])
    return model_2
```

Model Training:

```
import tensorflow as tf
model = buildModel()
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()
history = model.fit(x_train, y_train, epochs=30,
                    batch_size=128,
                    validation_split=ratio,
                    callbacks=callback_list)
model.save(filepath=model_path)
```

Bibliography

[1] Daydulo, Y.D., Thamineni, B.L. & Dawud, A.A. Cardiac arrhythmia detection using deep learning approach and time frequency representation of ECG signals. *BMC Med Inform Decis Mak* **23**, 232 (2023). https://doi.org/10.1186/s12911-023-02326-w

[2] Singh, A.K., Krishnan, S. ECG signal feature extraction trends in methods and applications. *BioMed Eng OnLine* **22**, 22 (2023). https://doi.org/10.1186/s12938-023-01075-1

[3] Sun, A.; Hong, W.; Li, J.; Mao, J. An Arrhythmia Classification Model Based on a CNN-LSTM-SE Algorithm. Sensors 2024, 24, 6306. https://doi.org/10.3390/s24196306

[4] Bhatia, S.; Pandey, S.K.; Kumar, A.; Alshuhail, A. Classification of Electrocardiogram Signals Based on Hybrid Deep Learning Models. Sustainability 2022, 14, 16572. https://doi.org/10.3390/ su142416572

[5] M. AlMahamdy and H. B. Riley, "Performance study of different denoising methods for ECG signals," in *Proc. of the 4th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (ICTH-2014)*, Athens, USA, 2014.

[6] Din S, Qaraqe M, Mourad O, Qaraqe K, Serpedin E. ECG-based cardiac arrhythmias detection through ensemble learning and fusion of deep spatial-temporal and long-range dependency features. Artif Intell Med. 2024 Apr;150:102818. doi: 10.1016/j.artmed.2024.102818. Epub 2024 Feb 24. PMID: 38553158.

[7] H. A. Shah, F. Saeed, M. Diyan, N. A. Almujally, and J.-M. Kang, "ECG-TransCovNet: A hybrid transformer model for accurate arrhythmia detection using Electrocardiogram signals," *CAAI Transactions on Intelligence Technology*, 2023. DOI: 10.1049/cit2.12293

[8] Automatic detection for cardiac arrhythmia based on ECG and deep learning approaches. Li, Z. (Author). 1 Aug 2024

[9] S. Chatterjee, R. S. Thakur, R. N. Yadav, L. Gupta, and D. K. Raghuvanshi, "Review of noise removal techniques in ECG signals," *IET Signal Processing*, E-first, Sept. 15, 2020. DOI: 10.1049/iet-spr.2020.0104.

[10] Kutlu, Yakup & Kuntalp, D.. (2011). Feature extraction for ECG heartbeats using higher order statistics of WPD coefficients. Computer methods and programs in biomedicine. 105. 257-67. 10.1016/j.cmpb.2011.10.002.

[11] Karoui, Hend & Hamza, Sihem & Benayed, Yassine. (2024). Deep Learning for Cardiac Diseases Classification. 10.1007/978-3-031-70816-9_14.

[12] J. Jiang, H. Zhang, D. Pi, and C. Dai, "A novel multi-module neural network system for imbalanced heartbeats classification," *College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics*, Nanjing, China.

[13] Feng, Jianchao & Si, Yujuan & Sun, Meiqi & Zhang, Yu. (2024). A category incremental continuous learning model for imbalance arrhythmia detection. Measurement Science and Technology. 35. 10.1088/1361-6501/ad7e46.

[14] Bernhard, Stefan, Brensing, Andreas and Witte, Karl-Heinz. "6 Applications and Methods in Biosignal Processing". *Biosignal Processing: Fundamentals and Recent Applications with MATLAB ®*, Berlin, Boston: De Gruyter Oldenbourg, 2022, pp. 213-288. https://doi.org/10.1515/9783110736298-006

[15] Shahin Vakilinia, Xinyao Zhang and Dongyu Qiu. ’Analysis and Optimization of Big-Data Stream Processing.’ Department of Electrical and Computer Engineer- ing, Concordia University, Montreal, QC, Canada. : IEEE Global Communications Conference (GLOBECOM), 2016.

[16] Libina Rose Sebastian, Sheeba Babu, and Dr. Jubilant J Kizhakkethottam. ’Challenges with Big Data Mining: A Review.’ Dept. of Computer Science and En- gineering, St. Joseph’s College of Engineering and Technology, Palai, Kerala, India : International Conference on Soft-Computing and Network Security (ICSNS), 2015.

[17] Katal N, Gupta S, Verma P, Sharma B. Deep-Learning-Based Arrhythmia Detection Using ECG Signals: A Comparative Study and Performance Evaluation. Diagnostics (Basel). 2023 Dec 5;13(24):3605. doi: 10.3390/diagnostics13243605. PMID: 38132188; PMCID: PMC10742760.

[18] A. Chandrasekar, D. D. Shekar, A. C. Hiremath, and K. Chemmangat, "Detection of arrhythmia from electrocardiogram signals using a novel Gaussian assisted signal smoothing and pattern recognition," *Biological Signal Processing and Control*, vol. 2021, art. no. 103469, 2021. DOI: 10.1016/j.bspc.2021.103469.

Biodata

Name: Vinodh S
Mobile No.: 6382213940
E-mail: vinodhencode@gmail.com
Permanent Address: No. 4 VGP Saraswati Nagar 1st street Rajkilpakkam Chennai - 600073

Name: Navaneth A
Mobile No.: 7358247423
E-mail: navaneth24@gmail.com
Permanent Address: G1, Ruby Enclave, 29, Bharatwajar street, East Tambaram, Chennai - 600059

Name: V M Adhityaa
Mobile No.: 8072007617
E-mail: madhusudhananadhityaa@gmail.com
Permanent Address: K303, Central Park South, Shollinganallur, Chennai - 600119